

Kontext und Kernel-Preemption

Ausführungskontexte, Unterbrechbarkeit, Locking und PREEMPT_RT

2.07

Der Kontext bestimmt, was Kernel-Code darf

BLOCKIEREN

Darf ich schlafen?

Mutex, Wait-Operationen und viele Speicher- oder Gerätezugriffe benötigen einen zulässigen Schlafkontext.

UNTERBRECHUNG

Bin ich preemptibel?

Die aktuelle Preemption-Sperre, IRQ-Situation und Kernelkonfiguration bestimmen die Unterbrechbarkeit.

DATENSCHUTZ

Wer greift noch zu?

Andere Tasks, CPUs, Hard-IRQs oder Bottom Halves erfordern unterschiedliche Synchronisation.

Ein gültiger Task allein reicht nicht: Auch ein Prozesskontext kann durch Locks oder IRQ-Sperren atomar sein.

Prozesskontext besitzt einen planbaren Task

SYSTEM CALL

Der aktuelle User-Task führt Kernel-Code im Auftrag einer Anwendung aus.

KERNEL-THREAD

Ein Kernel-Task arbeitet ohne gewöhnliches User-Space-Programm.

WORKER-THREAD

Normale Workqueue-Arbeit läuft im Prozesskontext eines kworker.

THREADED IRQ

Der IRQ-Thread ist planbar; sein Handler darf grundsätzlich schlafen.

Schlafen bleibt verboten, wenn der Pfad zusätzlich eine atomare Sperre oder eine andere Nichtblockierregel hält.

Hard-IRQ und Soft-IRQ sind keine normalen Tasks

HARD-IRQ · PRIMÄRER HANDLER

Sofort und kurz

Hardware quittieren, Ursache prüfen und weitere Arbeit auslösen. Kein Mutex, kein Warten, kein schlafender Buszugriff.

SOFT-IRQ · BOTTOM HALF

Konfiguration beachten

Auf Nicht-RT-Kernen meist atomar; auf PREEMPT_RT typischerweise threaded und preemptibel. WQ_BH bleibt nicht schlafbar.

Der Thread-Handler eines IRQ ist Prozesskontext; sein primärer Handler bleibt dagegen echter Hard-IRQ Code.

Atomare Abschnitte dürfen nicht blockieren

01 Typische atomare Auslöser

Hard-IRQ, `raw_spinlock_t`, `preempt_disable()` oder lokal deaktivierte Interrupts.

02 Unzulässige Operationen

`mutex_lock()`, Warten, `schedule()` und Geräte-APIs, die schlafen können.

03 Speicher besonders sorgfältig prüfen

GFP_KERNEL kann schlafen; unter PREEMPT_RT hilft GFP_ATOMIC im explizit atomaren Pfad nicht zuverlässig.

04 Fehler früh sichtbar machen

`might_sleep()` und Lock-Debugging können unerlaubtes Schlafen aufdecken.

RT-sicherer Entwurf: Speicher außerhalb echter atomarer Abschnitte reservieren und freigeben.

Interrupt, Preemption und Taskwechsel unterscheiden

HARDWAREEREIGNIS

Interrupt

Ein asynchrones Ereignis unterbricht die aktuelle CPU-Ausführung und startet einen IRQ-Pfad.

SCHEDULER-ENTSCHEIDUNG

Preemption

Ein laufender Task wird zugunsten eines anderen planbaren Tasks unterbrochen.

ZUSTANDSWECHSEL

Context Switch

Der Scheduler wechselt den aktiven Task; das kann freiwillig oder durch Preemption geschehen.

Nicht jeder Interrupt erzeugt einen Taskwechsel – und ein freiwilliger Taskwechsel ist keine erzwungene Preemption.

preempt_disable sperrt nicht die Interrupts

KRITISCHE ABSCHNITTE · VEREINFACHT

```
preempt_disable();  
/* Task bleibt auf der CPU; IRQs bleiben möglich. */  
use_this_cpu_data();  
preempt_enable();  
  
local_irq_save(flags);  
/* Nur lokale CPU-Interrupts sind deaktiviert. */  
local_irq_restore(flags);  
  
/* Beide Mechanismen haben unterschiedliche Aufgaben. */
```

01 Preemption

Verhindert den Scheduler-Wechsel des aktuellen Tasks, bis der Abschnitt endet.

02 Lokale IRQs

Blockiert Interrupts nur auf dieser CPU; andere CPUs laufen weiter.

Nesting zählt: Erst das passende letzte preempt_enable() macht den Abschnitt wieder preemptibel.

Preemption-Modelle priorisieren unterschiedliche Ziele

PREEMPT_NONE

Wenig erzwungene Kernel-Preemption; Fokus auf Durchsatz.

PREEMPT_VOLUNTARY

Zusätzliche freiwillige Rescheduling-Punkte.

PREEMPT / FULL

Unterbricht zulässigen Kernel-Code auch unfreiwillig.

PREEMPT_LAZY

Scheduler-gesteuert; vermeidet unnötig frühe Preemption.

PREEMPT_RT

Threaded IRQs und RT-Locks senken Scheduling-Latenzen.

PREEMPT_DYNAMIC erlaubt – sofern unterstützt – preempt=none, voluntary, full oder lazy beim Booten.

Ein Taskwechsel sichert und ersetzt CPU-Zustand

01 Rescheduling-Bedarf erkennen

Ein Task blockiert, wird verdrängt oder ein höherpriorer Task wird runnable.

02 Vorherigen Task sichern

Register, Stack-Kontext und schedulerrelevante Daten werden architekturabhängig gesichert.

03 Nächsten Task auswählen

Priorität, Scheduling-Klasse, CPU-Affinität und Runnable-Zustand beeinflussen die Auswahl.

04 Neuen Ausführungskontext laden

Register und Stack wechseln; der virtuelle Adressraum wird nur bei Bedarf gewechselt.

Auch Tasks mit gemeinsamem Adressraum besitzen getrennte Register-, Stack- und Scheduler-Zustände.

Lock-Typen behalten unter RT nicht dieselbe Semantik

LOCK	OHNE PREEMPT_RT	MIT PREEMPT_RT
mutex	kann schlafen	kann schlafen
spinlock_t	spinnt; sperrt Preemption	rt_mutex-basiert; preemptibel
raw_spinlock_t	echter Spinlock; atomar	bleibt echter Spinlock
local_lock_t	expliziter lokaler Schutz	per-CPU spinlock_t

Unter RT deaktiviert `spin_lock_irqsave()` mit gewöhnlichem `spinlock_t` die Hardwareinterrupts nicht.

PREEMPT_RT verschiebt Arbeit in planbare Threads

01 Interrupts werden weitgehend threaded

Die meisten IRQ-Pfade werden planbar; markierte Ausnahmen und primäre Handler bleiben hardirq.

02 Normale Spinlocks können blockieren

`spinlock_t` nutzt RT-Mutex-Semantik und Priority Inheritance statt einer dauerhaften Preemption-Sperre.

03 Bottom Halves benötigen expliziten Schutz

`local_bh_disable()` allein ist kein tragfähiger Ersatz für eine RT-kompatible per-CPU-Sperre.

04 Echte Atomik bleibt bewusst klein

`raw_spinlock_t`, deaktivierte IRQs und explizit gesperrte Preemption verlängern weiterhin Latenzen.

Priority Inheritance hilft nicht gegen einen Codepfad, der Preemption oder Interrupts tatsächlich sperrt.

Schlafende Arbeit gehört in den passenden Thread

PRIMÄRER IRQ-HANDLER

Nur quittieren

Geräteursache prüfen, notwendige Hardware bestätigen und `IRQ_WAKE_THREAD` zurückgeben.

THREADED IRQ

Gerät bedienen

I²C/SPI-Zugriffe, geeignete Mutex-Sperren und sonstige blockierende Arbeit im IRQ-Thread.

NORMALE WORKQUEUE

Asynchron arbeiten

kworker übernimmt unabhängige Arbeit im Prozesskontext. Achtung: `WQ_BH` ist nicht schlafbar.

IRQ-Thread oder reguläre Workqueue auswählen; ein `WQ_BH`-Work-Item ersetzt keinen schlafbaren Worker.

Ein threaded IRQ erlaubt sicheren I²C-Zugriff

SCHEMATISCHER TREIBERCODE · FEHLERPRÜFUNG AUSGELASSEN

```
static irqreturn_t sensor_irq(int irq, void *data)
{
    return IRQ_WAKE_THREAD; /* hardirq: kein Mutex */
}

static irqreturn_t sensor_thread(int irq, void *data)
{
    struct sensor *sensor = data;

    mutex_lock(&sensor->lock);
    sensor_read_i2c(sensor); /* darf schlafen */
    mutex_unlock(&sensor->lock);
    return IRQ_HANDLED;
}

request_threaded_irq(irq, sensor_irq, sensor_thread, IRQF_ONESHOT, "sensor", sensor);
```

Primärer Handler: atomar und minimal. Thread-Handler: planbar, mit zulässigem Mutex und I²C-Zugriff.

Konfiguration und IRQ-Threads praktisch prüfen

NUR LESEN · TRACEFS UND CONFIG SIND SYSTEMABHÄNGIG

```
$ uname -a
$ grep '^CONFIG_PREEMPT' "/boot/config-$(uname -r) "
$ cat /proc/cmdline
$ ps -eLo pid,tid,cls,rtprio,stat,comm | head
$ head /proc/interrupts
$ head /proc/softirqs
$ sudo cat /sys/kernel/tracing/available_tracers
$ sudo cat /sys/kernel/tracing/available_events
```

01 Build + Boot

CONFIG_PREEMPT*
prüfen und preempt= aus
der
Kernel-Kommandozeile
abgleichen.

02 Laufzeit

IRQ-Threads,
Scheduling-Klassen und
verfügbare Tracer ohne
Änderung ansehen.

Tracefs, Kernel-Konfigurationsdatei und Leserechte sind nicht auf jedem System verfügbar.

Kontext zuerst prüfen, dann Lock und API wählen

01 Ausführungskontext benennen

Prozess, IRQ-Thread, normaler Worker, Hard-IRQ, Softirq oder WQ_BH?

02 Blockieren und Preemption prüfen

Sind IRQs oder Preemption gesperrt? Hält der Pfad einen echten raw_spinlock?

03 Datenbesitz und RT-Regeln festlegen

Wer konkurriert auf welcher CPU? Welche Semantik hat der gewählte Lock unter PREEMPT_RT?

04 Latenz mit geeigneten Traces messen

sched_switch, IRQ-Events und verfügbare preemptoff-/irqsoff-Tracer gezielt untersuchen.

Robuste Treiber halten Hard-IRQ-Code kurz, verschieben Schlafen in Threads und bleiben RT-kompatibel.